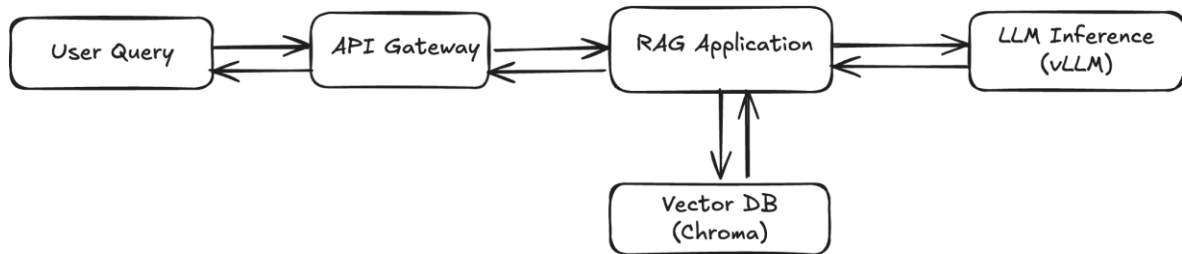


Part A – System Design

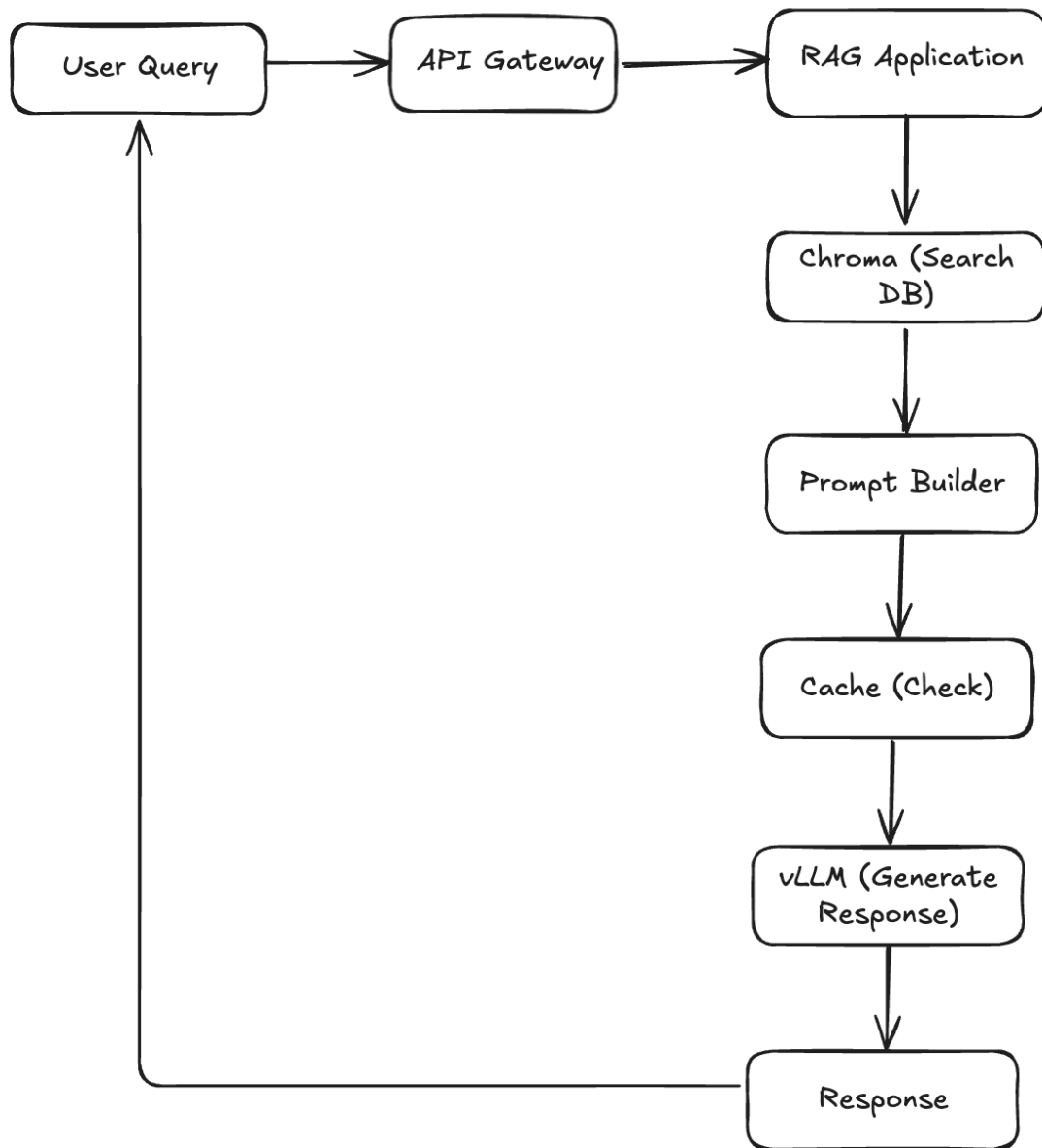
Section 1: System Architecture



Components:

- API Gateway: Entry point, Authentication, Rate Limiting
- RAG Application: FastAPI orchestration (embed -> search -> prompt -> cache -> vLLM)
- vLLM: LLM inference server on GPU
- Chroma: Vector database for similarity search

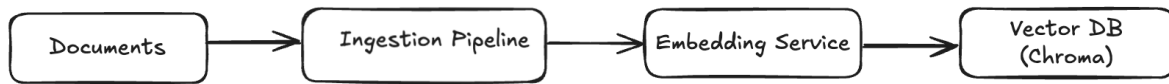
User Query Flow



Components:

- API Gateway: Entry point, auth, rate limiting
- RAG Application: FastAPI orchestration (embed -> search -> prompt -> cache -> vLLM)
- vLLM: LLM inference server on GPU
- Chroma: Vector database for similarity search
- Prompt Builder: System instructions + context + question
- Cache Layer: Frequent Q&A for instant retrieval

Ingestion Flow (Monthly Batch):



Components:

Ingestion Pipeline: Parses PDFs/Word/txt -> extracts text -> chunks -> embeds -> stores in vector DB

Embedding Service: Converts text to embeddings using a small on-prem model

Vector DB (Chroma): Stores embeddings; enables fast similarity search

Section 2: Key Decisions and Tradeoffs

Decision: LLM Inference via vLLM.

Tradeoff: Uses more VRAM for KV cache but enables Continuous Batching, ensuring the "snappy" response time for 20 concurrent users by processing requests in parallel rather than a first-in-first-out queue.

Chunk size: 512 tokens with 50-token overlap - Larger = better context but slower retrieval; overlap preserves context across chunks

Embedding model: Small (~100M params) - Smaller = faster but less accurate embeddings; choice balances speed and quality

Retrieval: Top 5 chunks, no reranking - More chunks = better recall but slower; reranking adds 100-200ms latency

Caching: In-memory dict - Simple but cleared on restart. Redis would persist but adds ops complexity

Document updates: Upsert strategy (update if exists, add if new) - Instead of append-only which would duplicate content

Indexing Strategy: Chroma with Flat Index - Use a flat index for 2,000 documents.

Tradeoff: Slightly higher search time than Hierarchical Navigable Small World (HNSW) at massive scales, but provides 100% recall and eliminates the stale index complexity often found in approximate nearest neighbor (ANN) graphs at this volume.

Document Updates

Monthly batch update uses upsert strategy (update if exists, add if new).

Chroma assigns unique IDs to chunks, so updated documents overwrite old versions.

This ensures retrieval does not return outdated content alongside new content.

Assumptions (not in MVP):

- Input validation handled by API Gateway (query sanitization, length limits)
- System prompt fixed for MVP (could be configurable with more time)
- Cache cleared on restart (in-memory dict)

What I Chose NOT to Do

- Real-time document sync: Monthly batch sufficient; real-time adds infrastructure complexity
- Reranking layer: Adds latency; top 5 adequate for query response time requirement
- Redis persistence: In-memory cache adequate for small team; adds ops/maintenance overhead
- Multi-tenant DB isolation: Handled at API Gateway with department headers; sufficient for shared endpoint

Section 3: Post-Deployment Monitoring

P50/P95/P99 latency: Ensures "snappy" response; alerts if degraded

Cache hit rate: Indicates repeat queries; guides cache sizing

Retrieval recall@k: Measures if relevant docs are being retrieved

LLM error rate: Catches infrastructure issues

User feedback: Thumbs up/down per response; aggregate weekly to spot trends

Section 4: Production Failure Mode

Context Pollution: Retrieved chunks contain contradictory info from different document versions (e.g., 2025 HR policy and 2026 HR policy both retrieved).

LLM receives conflicting contexts and may answer incorrectly even though source documents are correct.

Why only surfaces in production:

- Development uses small, curated test sets (no version conflicts)
- At 2,000 documents scale, probability of version collisions increases
- Monthly updates introduce new version before old version removed

How to identify in production:

- Users report inconsistent answers to same question over time
- Audit retrieval results: query same question and inspect retrieved chunks
- Check chunk metadata for version conflicts

Mitigation:

- Add document version date to chunk metadata
- Filter by recency: only retrieve chunks from last 2 versions
- Or delete old version chunks before adding new